

SESSION – 9

Top K Frequent Elements

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int element : nums){
            map.put(element, map.getOrDefault(element,0)+1);
        }

        ArrayList<Map.Entry<Integer,Integer>> arr =
            new ArrayList<>(map.entrySet());

        arr.sort((a,b)->b.getValue()-a.getValue());

        int[] last = new int[k];

        for(int i=0;i<k;i++){
            last[i]=arr.get(i).getKey();
        }

        return last;
    }
}
```

The screenshot displays a coding platform interface with three main sections: Submissions, Testcase, and Code.

- Submissions:** Shows the problem title "347. Top K Frequent Elements" with a "Solved" status. It includes the problem description: "Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order." Example 1: Input: `nums = [1,1,1,2,2,3]`, `k = 2`; Output: `[1,2]`. Example 2: Input: `nums = [1]`, `k = 1`; Output: `[1]`. Example 3: Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`.
- Testcase:** Shows "Accepted" status with a runtime of 1 ms. It lists three cases: Case 1, Case 2, and Case 3. The input for Case 1 is `nums = [1,1,1,2,2,3]` and `k = 2`. The output is `[1,2]`. The expected output is also `[1,2]`.
- Code:** Displays the Java code for the solution, which matches the code provided in the previous block. The code uses a `HashMap` to count frequencies, an `ArrayList` to store the entries, and a `sort` method to order them by frequency. It then extracts the top `k` elements into an array.